

NOTE:

- These are not exact midterm questions. These questions are intended to review the material.

1 Comparing Functions

For each of the following pairs of functions, write down the symbol that most precisely describes the asymptotic relation between $f(n)$ and $g(n)$; i.e., if $f(n) = x(g(n))$ where $x \in \{o, O, \Theta, \Omega, \omega\}$, then write down x under the "Relation" column in the corresponding row (an example is provided on the 0th row). You do not need to justify your answer (but try one for practice). Make sure to clearly distinguish O and o .

#	$f(n)$	$g(n)$	Relation
0	$n^4 + 3$	$3n^4$	Θ
1	$n^{0.5}$	n^{-4}	
2	$\log(n^4)$	$\log(n^{0.5})$	
3	$4n^3$	$3n^4$	
4	$n^{-3} + 0.5$	4	
5	$4^n + 3$	$3^n + 4$	

Key Tips to Solve**1. Identify dominant terms**

- For polynomials, focus on the highest-power term.
- For exponentials, compare bases.
- For logarithms, reduce constants (e.g., $\log(n^4) = 4 \log n$).

2. In many problems, constants don't matter

- n^4 and $3n^4$ are Θ of each other.
- But a shift in exponent or base can drastically change the relationship: n^3 vs. n^4

3. Use ratio or limit tests

- (a) If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, then $f(n) = o(g(n))$.
- (b) If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ is a nonzero finite constant, then $f(n) = \Theta(g(n))$
- (c) If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$, then $f(n) = \omega(g(n))$.

4. Always match the correct notation

- (a) o/O : means “eventually smaller or bounded.”
- (b) ω/Ω : means “eventually larger or bounded below.”
- (c) Θ : means “same growth up to constant factors.”

0. For large n , both behave like n^4 . Their ratio

$$\frac{n^4 + 3}{3n^4} \rightarrow \frac{1}{3}$$

is a nonzero constant, so $f(n) = \Theta(g(n))$.

1. Here

$$\frac{n^{0.5}}{n^{-4}} = n^{4.5} \rightarrow \infty$$

which shows $f(n)$ grows faster than $g(n)$. In “little-omega” terms, $f(n) = \omega(g(n))$.

2. Since $\log(n^4) = 4 \log n$ and $\log(n^{0.5}) = 0.5 \log n$, their ratio tends to

$$\frac{4}{0.5} = 8$$

So $f(n) = \Theta(g(n))$.

3. The ratio

$$\frac{4n^3}{3n^4} = \frac{4}{3n} \rightarrow 0$$

so $f(n)$ is strictly smaller order than $g(n)$: $f(n) = o(g(n))$.

4. As $n \rightarrow \infty$, $n^{-3} \rightarrow 0$, so $f(n) \rightarrow 0.5$. Thus

$$\frac{f(n)}{g(n)} \rightarrow \frac{0.5}{4} = 0.125$$

a nonzero constant. Therefore $f(n) = \Theta(g(n))$.

5. For large n , 4^n outgrows 3^n . So,

$$\frac{4^n + 3}{3^n + 4} \approx \frac{4^n}{3^n} = \left(\frac{4}{3}\right)^n \rightarrow \infty.$$

Therefore $f(n)$ is larger order than $g(n)$: $f(n) = \omega(g(n))$

2 Solving Recurrences

Provide a tight asymptotic solution for each of the following recurrences. If Master Method applies, also state the case of the master method that applies. If it does not, put "N/A". The first row is provided as an example. You do not need to justify your answer (but try one for practice).

#	Recurrence	Solution	Case
0	$T_0(n) = 9T(\frac{n}{9}) + \Theta(n)$	$\Theta(n \log n)$	2
1	$T_1(n) = 3T_1(\frac{n}{9}) + \Theta(n)$	$\Theta(n)$	3
2	$T_2(n) = 9T_2(\frac{n}{3}) + \Theta(n)$	$\Theta(n^2)$	1
3	$T_3(n) = 9T_3(\frac{n}{3}) + \Theta(n^2)$	$\Theta(n^2 \log n)$	2
4	$T_4(n) = T_4(n - 1000) + \Theta(n^2)$	$\Theta(n^3)$	N/A
5	$T_5(n) = T_5(\frac{n}{3}) + \Theta(1)$	$\Theta(\log n)$	2

1. $T_1(n) = 3T_1(\frac{n}{9}) + \Theta(n)$
 - $a = 3, b = 9, f(n) = \Theta(n)$
 - Compare $f(n) = \Theta(n)$ with $n^{\log_9 3} = n^{1/2}$
 - Since $f(n)$ dominates, Case 3 applies.

Check the regularity condition:

$$af(\frac{n}{b}) = 3f(\frac{n}{9}) = 3\frac{n}{9} = \frac{n}{3} \leq cf(n) = cn$$

for some $c \geq \frac{1}{3}$. Therefore regularity holds.

Solution: $T_1(n) = \Theta(n)$

2. $T_2(n) = 9T_2(\frac{n}{3}) + \Theta(n)$
 - $a = 9, b = 3, f(n) = \Theta(n)$
 - Compare $f(n) = \Theta(n)$ with $n^{\log_3 9} = n^2$
 - Since n^2 dominates, Case 1 applies.

Solution: $T_2(n) = \Theta(n^2)$

3. $T_3(n) = 9T_3(\frac{n}{3}) + \Theta(n^2)$
 - $a = 9, b = 3, f(n) = \Theta(n^2)$
 - Compare $f(n) = \Theta(n^2)$ with $n^{\log_3 9} = n^2$

- Since both are the same, Case 2 applies.

Solution: $T_3(n) = \Theta(n^2 \log n)$

4. $T_4(n) = T_4(n - 1000) + \Theta(n^2)$

This is a recurrence of the form $T(n) = T(n - c) + f(n)$, which expands to:

$$T(n) = \sum_{i=0}^{n/1000} \Theta(n^2)$$

which simplifies to $\Theta(n^3)$. The Master Method does not apply here.

5. $T_5(n) = T_5(\frac{n}{3}) + \Theta(1)$

- $a = 1, b = 3, f(n) = \Theta(1)$
- Compare $f(n) = \Theta(1)$ with $n^{\log_3 1} = n^0 = 1$
- Since both are equal, Case 2 applies.

Solution: $T_5(n) = \Theta(\log n)$

3 Create Algorithm

Tom has a set, A , of n nuts and a set, B , of n bolts, such that each nut has a unique matching bolt. Unfortunately, the nuts in A all look the same, and the bolts in B all look the same, as well. The only comparison that Tom can make is to take a nut-bolt pair (a, b) , such that a is in A and b is in B , and test if the threads of a are larger, smaller, or a perfect match with the threads of b . Describe an efficient algorithm for Tom to match up all of his nuts and bolts.

General Idea

1. Choose a pivot from set A (nuts) and use it to partition set B (bolts), ensuring bolts are divided into those smaller and larger than the pivot nut.
2. Find the matching bolt for the pivot nut during the partitioning of B . This bolt is now the pivot bolt.
3. Partition set A (nuts) using the pivot bolt found in step 2, separating nuts into smaller and larger than the pivot bolt.
4. Recursively apply these steps to the subsets of nuts and bolts created by the partitioning until all nuts and bolts are matched.

Pseudocode

Runtime Analysis

The runtime of this algorithm can be analyzed using the Master Theorem. Each time we partition the sets of nuts and bolts, we effectively divide the problem into two halves, leading to a recurrence relation of the form:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

Here, $\Theta(n)$ accounts for the partitioning time of nuts and bolts.

Algorithm 1 Organize the bolts and nuts to pairs where Bolt[i] matches Nut[i]

```
1: function MATCHNUTSANDBOLTS(Nuts, Bolts, low, high)
2:   if  $low < high$  then
3:     pivotBoltIndex  $\leftarrow$  PARTITION(Bolts, low, high, Nuts[high])     $\triangleright$  Partition Bolts
        around Nuts[high]
4:     PARTITION(Nuts, low, high, Bolts[pivotBoltIndex])     $\triangleright$  Partition Nuts around the
        bolt pivot
5:     MATCHNUTSANDBOLTS(Nuts, Bolts, low, pivotBoltIndex-1)
6:     MATCHNUTSANDBOLTS(Nuts, Bolts, pivotBoltIndex+1, high)
7:   end if
8: end function
```

According to the Master Theorem, since $n^{\log_2 2} = n$, this is case 2, which means the runtime is $O(n \log n)$.

Proof of Correctness

We prove the correctness of the algorithm using induction.

Base Case: When the size of the subsets of nuts and bolts is 1 (i.e., $n = 1$), the single nut will match the single bolt, satisfying the base case.

Inductive Step: Assume the algorithm correctly matches all nuts and bolts for any set of size k . When the size is $k + 1$, the algorithm chooses a pivot nut, partitions the bolts (which works as partition is defined in the textbook), finds the matching bolt, and then uses it to partition the nuts. The pivot nut and bolt are correctly matched. By the inductive hypothesis, since the subsets of nuts and bolts less than and greater than the pivot are of size less than or equal to k , they will also be correctly matched.

By induction, this proves the algorithm will correctly match all nuts and bolts for arrays size n .